GIÁO TRÌNH GIẢI PHẪU TOÀN DIỆN MÃ NGUỒN VÀ MÔ PHỎNG THUẬT TOÁN BẰNG TAY (HAND SIMULATION)

ĐỒ ÁN 1: PACMAN AI SEARCH & TRÒ CHƠI ĐỐI KHÁNG

Học phần: Biểu diễn tri thức và tìm kiếm nâng cao (UET.AI2014) — Trường Đại học Công nghệ (VNU-UET)
Nhóm thực hiện: Nhóm 07
Mục tiêu tài liệu: Giúp người đọc dù **chưa từng biết lập trình** cũng có thể hiểu tường tận từng dòng lệnh, từng cú pháp Python, nắm rõ lý do thiết kế và **tự cầm bút nháp tay từng bước** hoạt động của mọi thuật toán trên giấy.

MỤC LỤC TỔNG QUAN

- PHẦN 1: HƯỚNG DẪN CÀI ĐẶT MÔI TRƯỜNG & KHỞI CHẠY TỪ A ĐẾN Z
  - 1.1. Chuẩn bị Python và Tạo Môi trường ảo (`.venv`)
  - 1.2. Cài đặt Thư viện phụ thuộc (`requirements.txt`)
  - 1.3. Chi tiết 6 Cách Khởi chạy Chương trình
- PHẦN 2: BẢNG QUY ƯỚC BẢN ĐỒ & TOÀN BỘ CÁC KỊCH BẢN THỬ NGHIỆM
- PHẦN 3: GIẢI PHẪU TẦNG DỮ LIỆU & KIỂM TRA HỢP LỆ (TỪNG DÒNG CODE + CÚ PHÁP)
  - 3.1. File `src/utils/map_loader.py` — Đọc File Bản Đồ
  - 3.2. File `src/utils/map_check_valid.py` — Hàng Rào Kiểm Tra Tính Hợp Lệ
  - 3.3. File `src/utils/map_info.py` — Trích Xuất Thực Thể & Trọng Số Địa Hình
- PHẦN 4: GIẢI PHẪU MÔ HÌNH TRẠNG THÁI `src/pacman/game_state.py` (TRÁI TIM HỆ THỐNG)
  - 4.1. Trích dẫn Toàn bộ Mã Nguồn `game_state.py`
  - 4.2. Giải phẫu Từng Cú pháp và Dòng lệnh
  - 4.3. Mô phỏng "Nháp tay" Cơ chế Bất biến (Immutability) & Tính Băm Hash
- PHẦN 5: GIẢI PHẪU CÁC THUẬT TOÁN TÌM KIẾM CỔ ĐIỂN & MÔ PHỎNG NHÁP TAY
  - 5.1. File `src/search/pm_search_problem.py` — Định Nghĩa Bài Toán
  - 5.2. File `src/search/dfs.py` (Depth-First Search) — Kèm Bảng Nháp Tay
  - 5.3. File `src/search/bfs.py` (Breadth-First Search) — Kèm Bảng Nháp Tay
  - 5.4. File `src/search/ucs.py` (Uniform-Cost Search) — Kèm Bảng Nháp Tay Mê Cung Đầm Lầy
  - 5.5. File `src/search/greedy.py` (Greedy Search) — Kèm Bảng Nháp Tay Bẫy Tham Lam
  - 5.6. File `src/search/a_star.py` (A* Search) — Giải mã biến `counter` & Chứng minh Toán học
  - 5.7. File `src/search/multi_goal_search.py` & Quy Hoạch Gom Thức Ăn Đa Chặng
- PHẦN 6: GIẢI PHẪU TÁC TỬ GHOST VÀ TÌM KIẾM ĐỐI KHÁNG
  - 6.1. File `src/ghost/ghost.py` — AI Truy Đuổi & Chống Va Chạm Nhập Ma
  - 6.2. File `src/logic_pacman_vs_ghost/adv_problem.py` & Giải phẫu Hàm `evaluate()`
  - 6.3. File `src/logic_pacman_vs_ghost/minimax.py` — Kèm Cây Vẽ Nháp Tay
  - 6.4. File `src/logic_pacman_vs_ghost/alpha_beta.py` — Kèm Cây Nháp Tay Cắt Tỉa
- PHẦN 7: GIẢI PHẪU GIAO DIỆN ARCADE UI & CƠ CHẾ ĐA LUỒNG THREADING
  - 7.1. File `ui/controller.py` — Đa luồng Không Nghẽn UI
  - 7.2. File `ui/renderer.py` & `ui/theme.py` — Thuật toán Vẽ Vector Hoạt ảnh
  - 7.3. File `ui/main.py` — Vòng Lặp Sự Kiện 60 FPS
- PHẦN 8: FILE KHỞI CHẠY TRUNG TÂM `main.py`
- PHẦN 9: BỘ 10 CÂU HỎI VẤN ĐÁP KINH ĐIỂN VÀ LỜI GIẢI MẪU ĐẠT ĐIỂM 10

PHẦN 1: HƯỚNG DẪN CÀI ĐẶT MÔI TRƯỜNG & KHỞI CHẠY TỪ A ĐẾN Z

1.1. Chuẩn bị Python và Tạo Môi trường ảo (`.venv`)

Dự án được viết hoàn toàn trên nền tảng **Python 3** (tương thích tối ưu trên Python 3.10, 3.11, 3.12, 3.13).

1. **Kiểm tra môi trường Python:** Mở cửa sổ dòng lệnh (Terminal/PowerShell) và gõ:

   ```
   python --version
   ```

   Nếu hiển thị phiên bản từ `Python 3.10.x` trở lên là đạt yêu cầu.

2. **Tạo môi trường ảo độc lập:** Tại thư mục gốc của dự án `P1_Nhom07_TEST_FIX`, chạy lệnh:

   ```
   python -m venv .venv
   ```

   Giải thích cơ chế: Lệnh này yêu cầu module `venv` của Python tạo ra một thư mục con mang tên `.venv`. Thư mục này chứa một bản sao độc lập của trình thông dịch Python và công cụ `pip`, giúp việc cài đặt thư viện không làm ảnh hưởng đến các phần mềm khác trên máy tính.

3. **Kích hoạt môi trường ảo:**

   - **Trên Windows PowerShell:**

     ```
     .\.venv\Scripts\Activate.ps1
     ```

     (Nếu gặp lỗi UnauthorizedAccess chặn chạy script trên Windows, gõ lệnh: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned` rồi thử lại).

   - **Trên Windows Command Prompt (cmd.exe):**

     ```
     .\.venv\Scripts\activate.bat
     ```

   - **Trên macOS / Linux:**

     ```
     source .venv/bin/activate
     ```

   Dấu hiệu nhận biết kích hoạt thành công: Đầu dòng lệnh sẽ xuất hiện tiền tố `(.venv)`.

1.2. Cài đặt Thư viện phụ thuộc (`requirements.txt`)

Dự án được tối ưu hóa tối đa về mặt kiến trúc, chỉ sử dụng duy nhất một thư viện bên thứ ba là `pygame` để hiển thị cửa sổ đồ họa tương tác. Mọi cấu trúc dữ liệu tìm kiếm (Heap, Queue, Deque, Threading) đều dùng thư viện chuẩn có sẵn của Python.

- **Vị trí file:** requirements.txt
- **Nội dung file:**

```
pygame==2.6.1
```

- **Lệnh cài đặt:**

```
pip install -r requirements.txt
```

1.3. Chi tiết 6 Cách Khởi chạy Chương trình

Cách 1: Giao diện đồ họa Vintage Arcade (Pygame UI) — Trực quan nhất

Dành cho người chấm hoặc khi thuyết trình đồ án để quan sát Pacman di chuyển mượt mà, Ghost đuổi bắt và xem Mini-map.

```
python main.py --ui
# Hoặc chạy trực tiếp module giao diện:
python -m ui.main
```

- **Thao tác chuột:** Click vào các Tab thuật toán (`DFS` , `BFS` , `UCS` , `GREEDY` , `A*` , `MINIMAX` , `ALPHABETA`) để đổi thuật toán; click `Maps` để chọn bản đồ; kéo thanh tốc độ từ `0.5x` đến `10x` .
- **Thao tác phím tắt:**
  - `Enter` : Bắt đầu chạy mô phỏng (Start).
  - `Space` : Tạm dừng / Tiếp tục (Pause / Resume).
  - `S` : Đi từng bước một (Step-by-step) để phân tích chi tiết từng quyết định của AI.
  - `R` : Đặt lại trạng thái ban đầu (Reset).
  - `Esc` : Thoát ứng dụng.

Cách 2: Menu dòng lệnh tương tác trên Terminal (Interactive CLI)

Dành cho việc chạy trên máy chủ console không có card đồ họa màn hình.

```
python main.py
```

- Chương trình tự động quét thư mục `maps/`, đánh số thứ tự `[1]`, `[2]`, ... Người dùng chỉ cần gõ số để chọn map và chọn thuật toán.

Cách 3: Chạy trực tiếp một dòng lệnh (One-line CLI)

Dành cho hệ thống chấm bài tự động (Auto-grader):

```
# Tìm đường A* trên bản đồ tiêu chuẩn:
python main.py --map maps/test_map.txt --algo ASTAR

# Tìm đường trên bản đồ có đầm lầy và nước:
python main.py --map maps/water_and_swamp_map.txt --algo UCS

# Chế độ đối kháng Minimax (độ sâu 2):
python main.py --map maps/adversarial_small.txt --algo MINIMAX --depth 2

# Chế độ đối kháng Alpha-Beta tìm 1 nước đi tốt nhất:
python main.py --map maps/adversarial_small.txt --algo ALPHABETA --depth 2 --mode step
```

Cách 4: Chạy tự động toàn bộ Thực nghiệm Benchmark (Mục 5 Đề bài)

```
python -m src.utils.benchmark
```

- Tự động đo đạc thời gian, số bước, số node duyệt và xuất báo cáo khoa học vào thư mục `results/`.

Cách 5: Chạy bộ 30 Unit Tests tự động (Mục 6.1 Đề bài)

```
python -m unittest discover tests
```

- Chạy toàn bộ 30 ca kiểm thử bao quát toàn bộ hệ thống, kết quả kỳ vọng đạt `OK`.

Cách 6: Chạy script kiểm tra nhanh thuật toán `test_part.py`

```
python test_part.py
```

PHẦN 2: BẢNG QUY ƯỚC BẢN ĐỒ & TOÀN BỘ CÁC KỊCH BẢN THỬ NGHIỆM

Bảng quy chuẩn ký tự mê cung:

Ký tự	Tên thực thể	Chi phí bước vào (Cost)	Quy tắc ngữ nghĩa
`#`	Tường đá (Wall)	∞	Vật cản cố định, không thực thể nào được bước vào.
` `	Đường đi trống	1	Đường đi thông thoáng tiêu chuẩn.
`P`	Pacman Spawn	1	Vị trí xuất phát của Pacman (bắt buộc đúng 1 vị trí).
`.`	Thức ăn (Food)	1	Hạt thức ăn nhỏ, ăn được cộng 10 điểm.
`G`	Điểm đích (Goal)	1	Cửa thoát hiểm hoặc mục tiêu kết thúc bài toán.
`@`	Con ma (Ghost)	1	Vị trí xuất phát của tác tử ma (từ 0 đến nhiều con).
`~`	Vùng nước (Water)	**2**	Địa hình nước ngập, tốn gấp đôi chi phí di chuyển.
`S`	Đầm lầy (Swamp)	**3**	Địa hình đầm lầy bùn đặc, tốn gấp ba chi phí di chuyển.

Danh mục bản đồ kiểm thử tiêu biểu trong `maps/`:

1. `maps/test_map.txt` (9×15): Bản đồ chuẩn gồm tường dích dắc và 2 Ghost.
2. `maps/large_map.txt` (22×19): Bản đồ quy mô lớn với 3 Ghost và mê cung phức tạp.
3. `maps/water_and_swamp_map.txt` (5×12): Bản đồ địa hình không đồng nhất, kiểm chứng thực tế sự khác biệt giữa BFS và UCS.
4. `maps/dead_end_maze.txt`: Bản đồ chứa các ngõ cụt cực sâu, thử thách khả năng quay lui.
5. `maps/multi_goal_test.txt`: Bản đồ gom nhiều thức ăn rải rác.
6. `maps/no_solution_map.txt` (9×9): Bản đồ mục tiêu bị bít kín, kiểm tra thuật toán phát hiện vô nghiệm.
7. `maps/adversarial_small.txt` (11×21): Bản đồ đối kháng hẹp, kiểm tra né ma cự ly gần.
8. `maps/bad_map.txt`: Bản đồ lỗi định dạng, kiểm tra bộ lọc dữ liệu an toàn.

PHẦN 3: GIẢI PHẪU TẦNG DỮ LIỆU & KIỂM TRA HỢP LỆ (TỪNG DÒNG CODE + CÚ PHÁP)

3.1. File `src/utils/map_loader.py` — Đọc File Bản Đồ

- **Vị trí file:** src/utils/map_loader.py
- **Mã nguồn đầy đủ:**

```
from src.utils.map_check_valid import check_map

def load_map(file_path):
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            game_map = [line.rstrip("\r\n") for line in f if line.strip() != ""]
        if check_map(game_map) == False:
            print(f"Bản đồ không hợp lệ: {file_path}")
            return None
        return game_map
    except FileNotFoundError:
        print(f"Không tìm thấy file chứa map: {file_path}")
        return None
```

Giải thích cú pháp & Từng dòng lệnh cho người mới bắt đầu:

1. `from src.utils.map_check_valid import check_map`:
   - *Cú pháp:* Lệnh `import` lấy hàm `check_map` từ thư mục `src/utils/` để dùng trong file này.
   - *Ý nghĩa:* Tách biệt việc nạp file (I/O) và việc kiểm tra tính đúng đắn của dữ liệu.
2. `def load_map(file_path):`:
   - *Cú pháp:* `def` dùng để định nghĩa một hàm mang tên `load_map`, nhận tham số đầu vào là chuỗi đường dẫn tệp tin `file_path`.
3. `try: ... except FileNotFoundError:`:
   - *Cú pháp:* Khối bắt lỗi ngoại lệ (Exception Handling).
   - *Bản chất:* Nếu người dùng truyền một tên file không tồn tại (ví dụ `maps/khong_co.txt`), Python sẽ không làm sập (crash) chương trình mà nhảy xuống dòng `except`, in thông báo lỗi và trả về `None`.
4. `with open(file_path, "r", encoding="utf-8") as f:`:
   - *Cú pháp:* `open(..., "r")` mở file ở chế độ chỉ đọc (Read-only); `encoding="utf-8"` đảm bảo đọc đúng ký tự trên cả Windows và Linux; `with ... as f` là Context Manager tự động đóng file sau khi đọc xong để giải phóng bộ nhớ hệ điều hành.
5. `game_map = [line.rstrip("\r\n") for line in f if line.strip() != ""]`:

- *Cú pháp:* Kỹ thuật List Comprehension (tạo danh sách nhanh trong Python).
- *Phân tích chi tiết:*
  - `for line in f`: Đọc từng dòng văn bản từ file.
  - `line.rstrip("\r\n")`: **Điểm cực kỳ quan trọng!** Hàm `rstrip` chỉ cắt bỏ ký tự xuống dòng ở cuối hàng. **Tuyệt đối không dùng** `line.strip()` vì `strip()` sẽ xóa luôn cả những dấu cách `' '` ở đầu dòng, làm cho hàng đó bị thụt lề ngắn lại, làm hỏng toàn bộ tọa độ cột của bản đồ.
  - `if line.strip() != ""`: Lọc bỏ những dòng trống hoàn toàn (người dùng vô tình enter ở cuối file).

6. `if check_map(game_map) == False:`:
   - Gửi danh sách các hàng vừa đọc sang hàm kiểm tra. Nếu vi phạm, từ chối nạp ngay lập tức.

3.2. File `src/utils/map_check_valid.py` — Hàng Rào Kiểm Tra Tính Hợp Lệ

- **Vị trí file:** src/utils/map_check_valid.py
- **Mã nguồn đầy đủ:**

```
def check_map(game_map):
    if len(game_map) == 0:
        return False

    for line in game_map:
        if len(line) != len(game_map[0]):
            return False

    allowed_char = "#.PG@~S "
    for line in game_map:
        for char in line:
            if char not in allowed_char:
                return False

    p_count = 0
    g_count = 0
    food_count = 0
    for line in game_map:
        p_count += line.count("P")
        g_count += line.count("G")
        food_count += line.count(".")

    if p_count != 1:
        return False
    if g_count + food_count < 1:
        return False

    return True
```

Giải thích cú pháp & Từng dòng lệnh:

1. `if len(game_map) == 0: return False`:
   - Kiểm tra nếu danh sách bản đồ rỗng (file 0 bytes) thì báo lỗi ngay, tránh việc dòng tiếp theo truy cập `game_map[0]` bị lỗi chỉ số `IndexError`.
2. `for line in game_map: if len(line) != len(game_map[0]): return False`:
   - *Cú pháp:* Vòng lặp `for` duyệt qua từng dòng `line`. `len(line)` là số ký tự của dòng đó.
   - *Ý nghĩa hình học:* Mọi dòng trong bản đồ đều phải có độ dài bằng chính xác độ dài dòng đầu tiên `len(game_map[0])`. Nếu có một dòng thò ra hoặc thụt vào, ma trận sẽ bị méo và lập tức bị từ chối.
3. `allowed_char = "#.PG@~S "`:
   - Danh sách "bạch thư" (whitelist) các ký tự được phép xuất hiện. Nếu trong file có chữ cái lạ như `'X'`, `'A'`, `'1'`, vòng lặp lồng `for char in line: if char not in allowed_char:` sẽ phát hiện và trả về `False`.
4. `line.count("P")`:
   - Đếm số lần xuất hiện của chữ `'P'` trên mỗi dòng.
   - `if p_count != 1: return False`: Ràng buộc bài toán đơn tác tử: **bắt buộc phải có đúng 1 Pacman**. Nếu không có Pacman hoặc có 2 Pacman trở lên thì bản đồ không hợp lệ.

- `if g_count + food_count < 1: return False`: Bản đồ bắt buộc phải có ít nhất 1 mục tiêu (Goal hoặc Food) để tìm kiếm.

3.3. File `src/utils/map_info.py` — Trích Xuất Thực Thể & Trọng Số Địa Hình

- **Vị trí file:** src/utils/map_info.py
- **Mã nguồn đầy đủ:**

```
def get_map_info(game_map, return_foods=False):
    spawn = None
    goals = []
    ghosts = []
    foods = []
    for i in range(len(game_map)):
        for j in range(len(game_map[i])):
            char = game_map[i][j]
            if char == "P":
                spawn = (i, j)
            elif char == "G":
                goals.append((i, j))
            elif char == "@":
                ghosts.append((i, j))
            elif char == ".":
                foods.append((i, j))

    if return_foods:
        return spawn, goals, ghosts, foods

    if len(goals) == 0 and len(foods) > 0:
        return spawn, foods, ghosts

    return spawn, goals, ghosts

TERRAIN_COSTS = {
    "~": 2,  # nước
    "S": 3,  # đầm lầy
}

def get_tile_cost(position, game_map):
    r, c = position
    if 0 <= r < len(game_map) and 0 <= c < len(game_map[r]):
        char = game_map[r][c]
        return TERRAIN_COSTS.get(char, 1)
    return 1
```

Giải thích cú pháp & Từng dòng lệnh:

1. `spawn = (i, j)` : Tọa độ hàng i và cột j được lưu dưới dạng `tuple`. Tuple trong Python có tính chất bất biến (không sửa đổi được), do đó rất an toàn khi truyền qua các hàm.
2. `goals.append((i, j))` : Thêm tọa độ điểm đích vào danh sách `goals`.
3. `TERRAIN_COSTS = {"~": 2, "S": 3}` :
    - Từ điển tra cứu chi phí (Dictionary).
    - Ký tự sóng nước `'~'` tốn chi phí 2; ký tự đầm lầy `'S'` tốn chi phí 3.
4. `TERRAIN_COSTS.get(char, 1)` :
    - *Cú pháp:* Phương thức `.get(khóa, giá_trị_mặc_định)`. Nếu ký tự `char` có trong từ điển thì lấy giá trị tương ứng; nếu không có (ví dụ ô trống `' '` hay thức ăn `'.'`), hàm tự động trả về giá trị mặc định là `1`.
    - *Tốc độ:* Phép tra cứu từ điển này diễn ra trong thời gian tức thì $O(1)$.

PHẦN 4: GIẢI PHẪU MÔ HÌNH TRẠNG THÁI `src/pacman/game_state.py` (TRÁI TIM HỆ THỐNG)

4.1. Trích dẫn Toàn bộ Mã Nguồn `game_state.py`

- **Vị trí file:** src/pacman/game_state.py

```
from src.ghost.ghost import Ghost

class GameState:
    def __init__(self, pacman_position, ghost_positions, food, score, game_map, status="PLAYING"):
        self.pacman_position = pacman_position
        self.ghost_positions = ghost_positions
        self.food = food
        self.score = score
        self.game_map = game_map
        self.status = status

    def copy_pacman_position(self, new_position):
        return GameState(new_position, self.ghost_positions, self.food, self.score, self.game_map)

    def check_valid_position(self, position):
        r, c = position
        if r < 0 or r >= len(self.game_map):
            return False
        if c < 0 or c >= len(self.game_map[r]):
            return False
        if self.game_map[r][c] == "#":
            return False
        return True

    def move(self, action):
        r, c = self.pacman_position
        if action == "UP": r -= 1
        elif action == "DOWN": r += 1
        elif action == "LEFT": c -= 1
        elif action == "RIGHT": c += 1
        new_position = (r, c)
        if self.check_valid_position(new_position) == False:
            return None
        new_food = self.food.copy()
        new_score = self.score

        if new_position in new_food:
            new_food.remove(new_position)
            new_score += 10

        return GameState(new_position, self.ghost_positions, new_food, new_score, self.game_map)

    def __eq__(self, other):
        if isinstance(other, GameState) == False:
            return False
        return(
            self.pacman_position == other.pacman_position
            and self.ghost_positions == other.ghost_positions
            and self.food == other.food
            and self.score == other.score
        )

    def __hash__(self):
        return hash((
            self.pacman_position,
```

```
            tuple(self.ghost_positions),
            frozenset(self.food),
            self.score
        ))

    def move_ghost(self, ghost_index, action):
        ghost = Ghost(self.ghost_positions[ghost_index])
        new_position = ghost.get_next_position(action, self.game_map)
        if new_position is None:
            return None

        new_ghost_positions = self.ghost_positions.copy()
        new_ghost_positions[ghost_index] = new_position

        return GameState(
            self.pacman_position,
            new_ghost_positions,
            self.food,
            self.score,
            self.game_map
        )

    def get_pacman_successors(self):
        successors = []
        actions = ["UP", "DOWN", "LEFT", "RIGHT"]
        for action in actions:
            next_state = self.move(action)
            if next_state is not None:
                successors.append((next_state, action))
        return successors

    def get_ghost_successors(self, ghost_index):
        successors = []
        ghost = Ghost(self.ghost_positions[ghost_index])
        valid_actions = ghost.get_valid_actions(self.game_map)
        for action in valid_actions:
            next_state = self.move_ghost(ghost_index, action)
            if next_state is not None:
                successors.append((next_state, action))
        return successors

    def check_pacman_caught(self):
        return self.pacman_position in self.ghost_positions

    def step(self, pacman_action, target, is_final_goal=False):
        next_state = self.move(pacman_action)
        if next_state is None:
            return None

        # Pacman va chạm Ghost ngay sau khi di chuyển
        if next_state.pacman_position in next_state.ghost_positions:
            next_state.status = "LOSE"
            return next_state

        # Pacman tới đích cuối cùng
        if next_state.pacman_position == target and (is_final_goal or len(next_state.food) == 0):
```

```
            next_state.status = "WIN"
            return next_state

        # Cho từng Ghost di chuyển
        for ghost_index in range(len(next_state.ghost_positions)):
            ghost = Ghost(next_state.ghost_positions[ghost_index])
            other_ghosts = [
                pos for idx, pos in enumerate(next_state.ghost_positions) if idx != ghost_index
            ]

            ghost_action = ghost.choose_action(
                next_state.game_map,
                next_state.pacman_position,
                other_ghost_positions=other_ghosts
            )
            if ghost_action is None:
                continue
            next_state = next_state.move_ghost(ghost_index, ghost_action)
            if next_state is None:
                return None

            # Ghost bắt được Pacman sau khi di chuyển
            if next_state.pacman_position in next_state.ghost_positions:
                next_state.status = "LOSE"
                return next_state
        return next_state
```

4.2. Giải phẫu Từng Cú pháp và Dòng lệnh

1. **Hàm khởi tạo `__init__` :**
   - Đóng gói toàn bộ biến cục bộ thành thuộc tính đối tượng qua từ khóa `self.` :
     - `self.pacman_position` : Tọa độ `(r, c)` .
     - `self.ghost_positions` : Danh sách tọa độ các Ghost.
     - `self.food` : Tập hợp các vị trí thức ăn dạng `set` .
     - `self.score` : Điểm số của người chơi.
     - `self.status` : Trạng thái trò chơi (`"PLAYING"` , `"WIN"` , `"LOSE"`).
2. **Hàm chuyển trạng thái `move(self, action)` — Mô hình Bất biến (Immutability Pattern):**
   - *Nguyên tắc toán học:* Trong thuật toán tìm kiếm cây/đồ thị, khi thử nghiệm một nước đi, ta **không được phép làm biến đổi trạng thái hiện tại**.
   - `new_food = self.food.copy()` : Sao chép tập thức ăn ra một vùng nhớ mới. Nếu Pacman ăn thức ăn tại `new_position` , ta chỉ xóa trong `new_food` và tạo ra đối tượng `GameState` mới:

     ```
     return GameState(new_position, self.ghost_positions, new_food, new_score, self.game_map)
     ```

- Trạng thái ban đầu `self` hoàn toàn nguyên vẹn. Nhờ đó, thuật toán có thể quay lui (backtracking) an toàn mà không bị lỗi mất dữ liệu.

3. **Hai phương thức `__eq__` và `__hash__` — Bí mật của Graph Search $O(1)$:**
   - Để thuật toán A* hay BFS biết được một ô đã đi qua hay chưa, nó lưu trạng thái vào tập hợp `visited = set()`.
   - Trong Python, muốn đưa một đối tượng vào `set`, đối tượng đó phải có hai hàm:
     - `__eq__(self, other)`: Định nghĩa khi nào hai trạng thái được coi là trùng nhau. Tại đây, hai trạng thái bằng nhau khi cùng vị trí Pacman, cùng vị trí Ghost và cùng tập thức ăn.
     - `__hash__(self)`: Tạo một mã số nguyên đại diện cho trạng thái.
   - **Cú pháp ép kiểu:**

     ```
     tuple(self.ghost_positions)  # Ép kiểu list -> tuple
     frozenset(self.food)         # Ép kiểu set -> frozenset
     ```

     Tại sao phải ép kiểu? Vì `list` và `set` là đối tượng có thể thay đổi (mutable), Python cấm băm chúng. Ép sang `tuple` và `frozenset` (immutable) cho phép Python tính mã băm cực nhanh trong $O(1)$.

4.3. Mô phỏng "Nháp tay" Cơ chế Bất biến (Immutability) & Tính Băm Hash

Giả sử ta có mê cung mini 3×3:

```
Row 0: ###
Row 1: #P.#
Row 2: ###
```

- **Trạng thái ban đầu S_0:**
  - `pacman_position` = $(1, 1)$
  - `food` = ${(1, 2)}$
  - `score` = 0
- **Pacman thực hiện hành động `move("RIGHT")`:**
  1. Tính vị trí mới: hàng giữ nguyên $r=1$, cột tăng $1 \implies (1, 2)$.
  2. Kiểm tra `check_valid_position((1, 2))`: ô $(1, 2)$ là `.` (không phải `#`), hợp lệ!
  3. Tạo bản sao tập thức ăn: `new_food = {(1, 2)}`.
  4. Vì $(1, 2) \in new_food \implies$ xóa $(1, 2)$ khỏi `new_food` và cộng điểm: `new_score = 0 + 10 = 10`.
  5. Trả về trạng thái mới S_1:
     - S_1: `pacman_position` = $(1, 2)$, `food` = $\emptyset$ (rỗng), `score` = 10.

6. **Kiểm tra trạng thái cũ S_0 trên giấy nháp:**
    - S_0 vẫn có `pacman_position` = $(1, 1)$, `food` vẫn còn nguyên ${(1, 2)}$, `score` vẫn là 0. $\implies$ **Chứng minh tính bất biến thành công!**

PHẦN 5: GIẢI PHẪU CÁC THUẬT TOÁN TÌM KIẾM CỔ ĐIỂN & MÔ PHỎNG NHÁP TAY

5.1. File `src/search/pm_search_problem.py` — Định Nghĩa Bài Toán

- **Vị trí file:** src/search/pm_search_problem.py
- **Mã nguồn đầy đủ:**

```
from src.pacman.game_state import GameState
from src.utils.map_info import get_tile_cost

class PMSearchProblem:
    def __init__(self, spawn, target, game_map, cost_fn=None, heuristic_fn=None):
        self.pacman_spawn = spawn
        self.target = target
        self.game_map = game_map
        if cost_fn is None:
            self.cost_fn = lambda pos: get_tile_cost(pos, self.game_map)
        else:
            self.cost_fn = cost_fn
        self.heuristic_fn = heuristic_fn

    def get_pacman_spawn(self):
        return GameState(
            self.pacman_spawn, [], set(), 0, self.game_map
        )

    def reached_target(self, state: GameState):
        return state.pacman_position == self.target

    def expand_pacman_moves(self, state):
        successors = []
        actions = ["UP", "DOWN", "LEFT", "RIGHT"]

        for action in actions:
            next_state = state.move(action)
            if next_state is not None:
                step_cost = self.cost_fn(next_state.pacman_position) if self.cost_fn else 1
                successors.append((next_state, action, step_cost))

        return successors

    def measure_path_cost(self, actions):
        if not self.cost_fn:
            return len(actions)
        curr = self.get_pacman_spawn()
        total_cost = 0
        for act in actions:
            curr = curr.move(act)
            if curr:
                total_cost += self.cost_fn(curr.pacman_position)
        return total_cost

    def heuristic(self, state: GameState):
        if self.heuristic_fn is not None:
            return self.heuristic_fn(state)
        row, col = state.pacman_position
        target_row, target_col = self.target
        return abs(row - target_row) + abs(col - target_col)
```

Giải phẫu cú pháp & Dòng lệnh:

- `self.cost_fn = lambda pos: get_tile_cost(pos, self.game_map)` : Cú pháp hàm ẩn danh `lambda` trong Python. Nhận vào vị trí `pos` và tự động gọi `get_tile_cost` để lấy chi phí bước đi theo địa hình.
- `expand_pacman_moves(state)` : Sinh tập trạng thái kế thừa. Thử 4 hướng di chuyển; với mỗi hướng hợp lệ, trả về một bộ ba gồm: `(trạng thái kế tiếp, tên hành động, chi phí bước đi)` .
- `heuristic(state)` : Mặc định tính khoảng cách Manhattan theo công thức: $$h(n) = |r_n - r_G| + |c_n - c_G|$$

5.2. File `src/search/dfs.py` (Depth-First Search) — Kèm Bảng Nháp Tay

- **Vị trí file:** src/search/dfs.py
- **Mã nguồn:**

```
import time

class DFS:
    def __init__(self, problem):
        self.problem = problem
        self.expanded_count = 0
        self.time_used = 0

    def search(self):
        start_time = time.time()
        self.expanded_count = 0

        start = self.problem.get_pacman_spawn()
        stack = [(start, [])]
        visited = set()

        while len(stack) > 0:
            state, actions = stack.pop()

            if state in visited:
                continue
            visited.add(state)
            self.expanded_count += 1

            if self.problem.reached_target(state):
                self.time_used = time.time() - start_time
                return actions
            successors = self.problem.expand_pacman_moves(state)
            for next_state, next_action, cost in successors:
                new_action = actions + [next_action]
                stack.append((next_state, new_action))
        self.time_used = time.time() - start_time
        return None
```

Giải phẫu cú pháp & Hoạt động:

- `stack = [(start, [])]` : Cấu trúc dữ liệu ngăn xếp (LIFO - Last In First Out).
- `state, actions = stack.pop()` : Rút phần tử ở cuối danh sách (phần tử mới nhất vừa được đưa vào).
- `visited = set()` : Ngăn chặn duyệt lại các ô đã đi qua, loại trừ hiện tượng lặp vòng vô hạn khi mê cung có chu trình.

📝 Bảng mô phỏng "Nháp tay" từng bước DFS trên Mê cung mẫu:

Xét mê cung kích thước 4×4:

```
Cột:   0 1 2 3
Hàng 0: # # # #
Hàng 1: # P   #   P tại (1, 1)
Hàng 2: #   G #   G tại (2, 2)
Hàng 3: # # # #
```

Thứ tự sinh hành động trong mã nguồn: `["UP", "DOWN", "LEFT", "RIGHT"]`.

Bước	Hành động trong mã	Lấy ra từ Stack (`pop`)	Tập `visited` sau bước	Các node con sinh ra và đẩy vào `stack` (`append`)	Trạng thái Stack hiện tại (Đỉnh ở bên phải)
0	Khởi tạo	Chưa	$\emptyset$	Đẩy node gốc $(1, 1)$	`[((1, 1), [])]`
1	Vòng 1	`(1, 1), []`	${(1, 1)}$	UP: đâm tường `#` DOWN: $(2, 1)$ hợp lệ LEFT: đâm tường `#` RIGHT: $(1, 2)$ hợp lệ	`[((2, 1), [DOWN]), ((1, 2), [RIGHT])]`
2	Vòng 2 (Lấy đỉnh)	`(1, 2), [RIGHT]`	${(1, 1), (1, 2)}$	UP: đâm tường DOWN: $(2, 2)$ hợp lệ LEFT: $(1, 1)$ có trong visited RIGHT: đâm tường	`[((2, 1), [DOWN]), ((2, 2), [RIGHT, DOWN])]`
3	Vòng 3 (Lấy đỉnh)	`(2, 2), [RIGHT, DOWN]`	${(1, 1), (1, 2), (2, 2)}$	**CHẠM ĐÍCH GOAL!** Dừng thuật toán.	Trả về chuỗi hành động: `['RIGHT', 'DOWN']`

- **Kết luận nháp tay:** DFS tìm thấy đường đi gồm 2 bước: `['RIGHT', 'DOWN']`, số node mở rộng = 3.

5.3. File `src/search/bfs.py` (Breadth-First Search) — Kèm Bảng Nháp Tay

- **Vị trí file:** src/search/bfs.py
- **Mã nguồn:**

```
import time
from collections import deque

class BFS:
    def __init__(self, problem):
        self.problem = problem
        self.expanded_count = 0
        self.time_used = 0

    def search(self):
        start_time = time.time()
        self.expanded_count = 0

        start = self.problem.get_pacman_spawn()
        queue = deque()
        queue.append((start, []))
        visited = set()
        visited.add(start)

        while len(queue) > 0:
            state, actions = queue.popleft()
            self.expanded_count += 1

            if self.problem.reached_target(state):
                self.time_used = time.time() - start_time
                return actions

            successors = self.problem.expand_pacman_moves(state)

            for next_state, next_action, cost in successors:
                if next_state in visited:
                    continue

                visited.add(next_state)
                new_action = actions + [next_action]
                queue.append((next_state, new_action))

        self.time_used = time.time() - start_time
        return None
```

Giải phẫu cú pháp & Điểm khác biệt cốt lõi:

- `queue = deque()` và `queue.popleft()`: Hàng đợi FIFO (First In First Out). Node nào vào trước sẽ được duyệt trước.
- **Kỹ thuật đánh dấu `visited` sớm (Early Visited Marking):** Dòng 19: `visited.add(start)` và Dòng 35: `visited.add(next_state)` được thực hiện **ngay khi node con vừa sinh ra**, trước khi đẩy vào `queue`. *Tại sao?* Nếu đợi lúc lấy ra mới đánh dấu (như DFS), một node có thể được sinh ra bởi nhiều node cha khác nhau và bị đẩy trùng lặp vào hàng đợi nhiều lần, làm tốn bộ nhớ RAM theo cấp số nhân.

📝 Bảng mô phỏng "Nháp tay" từng bước BFS trên Mê cung mẫu:

(Dùng cùng mê cung 4×4 ở trên, Pacman tại $(1, 1)$, Goal tại $(2, 2)$).

Bước	Lấy ra từ Queue (`popleft`)	Tập `visited` hiện tại	Các node con sinh ra	Hàng đợi Queue sau bước (Đầu bên trái, Cuối bên phải)
0	Khởi tạo	${(1, 1)}$	Đẩy $(1, 1)$ vào queue	`[((1, 1), [])]`
1	`(1, 1), []`	${(1, 1), (2, 1), (1, 2)}$	Sinh $(2, 1)$ và $(1, 2)$ Đánh dấu visited cả hai!	`[((2, 1), [DOWN]), ((1, 2), [RIGHT])]`
2	`(2, 1), [DOWN]` (Lấy đầu)	${(1, 1), (2, 1), (1, 2), (2, 2)}$	Sinh $(2, 2)$ (Goal)! Đánh dấu visited $(2, 2)$!	`[((1, 2), [RIGHT]), ((2, 2), [DOWN, RIGHT])]`
3	`(1, 2), [RIGHT]` (Lấy đầu)	...	Không sinh thêm node mới vì các ô kề đã có trong `visited`	`[((2, 2), [DOWN, RIGHT])]`
4	`(2, 2), [DOWN, RIGHT]`	...	**CHẠM ĐÍCH GOAL!** Dừng thuật toán.	Trả về chuỗi: `['DOWN', 'RIGHT']`

- **Kết luận:** BFS duyệt theo từng mức độ sâu hình gợn sóng (sóng loang), luôn đảm bảo tìm được nghiệm có số bước đi ít nhất.

5.4. File `src/search/ucs.py` (Uniform-Cost Search) — Kèm Bảng Nháp Tay Mê Cung Đầm Lầy

- **Vị trí file:** src/search/ucs.py
- **Mã nguồn:**

```
import time
import heapq

class UCS:
    def __init__(self, problem):
        self.problem = problem
        self.expanded_count = 0
        self.time_used = 0

    def search(self):
        start_time = time.time()
        self.expanded_count = 0

        start = self.problem.get_pacman_spawn()
        queue = []
        counter = 0
        heapq.heappush(queue, (0, counter, start, []))
        visited = set()

        while len(queue) > 0:
            cost, _, state, actions = heapq.heappop(queue)

            if state in visited:
                continue

            visited.add(state)
            self.expanded_count += 1

            if self.problem.reached_target(state):
                self.time_used = time.time() - start_time
                return actions

            successors = self.problem.expand_pacman_moves(state)

            for next_state, next_action, move_cost in successors:
                new_cost = cost + move_cost
                new_actions = actions + [next_action]
                counter += 1
                heapq.heappush(queue, (new_cost, counter, next_state, new_actions))

        self.time_used = time.time() - start_time
        return None
```

📝 Bảng mô phỏng "Nháp tay" UCS trên Mê cung có Đầm lầy (`maps/water_and_swamp_map.txt` thu nhỏ):

Xét mê cung có địa hình đầm lầy:

```
Cột:     0 1 2 3
Hàng 0:  # # # #
Hàng 1:  # P S G #   P tại (1, 1), Đầm lầy S tại (1, 2) chi phí 3, Goal tại (1, 3)
Hàng 2:  #       #   Hàng 2 là đất bằng (chi phí 1)
Hàng 3:  # # # #
```

Vòng lặp	Node lấy ra từ Heap (`heappop` với cost nhỏ nhất)	Chi phí g	Tập `visited` sau bước	Các nhánh con sinh ra và đẩy vào Heap `(cost, node, actions)`
0	Khởi tạo	0	$\emptyset$	Đẩy gốc: `(0, (1, 1), [])`
1	Lấy `(1, 1)`	0	${(1, 1)}$	- Đi RIGHT vào Đầm lầy $(1, 2)$: chi phí $0 + 3 = 3$ - Đi DOWN vào Đất bằng $(2, 1)$: chi phí $0 + 1 = 1$
2	Lấy node rẻ nhất: `(2, 1)`	**1**	${(1, 1), (2, 1)}$	- Đi RIGHT vào Đất bằng $(2, 2)$: chi phí $1 + 1 = 2$
3	Lấy node rẻ nhất: `(2, 2)`	**2**	${(1, 1), (2, 1), (2, 2)}$	- Đi RIGHT vào Đất bằng $(2, 3)$: chi phí $2 + 1 = 3$ - Đi UP vào Đầm lầy $(1, 2)$: chi phí $2 + 3 = 5$
4	Hiện tại trong Heap có 2 node cùng cost = 3: `(1, 2)` (lội đầm lầy) và `(2, 3)` (đi vòng). Node `(1, 2)` sinh trước nên lấy ra	3	${(1, 1), (2, 1), (2, 2), (1, 2)}$	- Đi RIGHT vào Goal $(1, 3)$: chi phí $3 + 1 = 4$
5	Lấy node rẻ nhất: `(2, 3)`	**3**	${(1, 1), ..., (2, 3)}$	- Đi UP vào Goal $(1, 3)$: chi phí $3 + 1 = \mathbf{4}$
6	Lấy `(1, 3)` (Goal) qua đường vòng phía dưới	**4**	...	**CHẠM ĐÍCH!** Lựa chọn đường vòng: `['DOWN', 'RIGHT', 'RIGHT', 'UP']`

- **Ý nghĩa rút ra từ bài toán nháp tay:** Nếu đi thẳng qua đầm lầy, chi phí là $0 + 3 + 1 = 4$ (2 bước). Đi vòng mất 4 bước nhưng chi phí cũng là $1 + 1 + 1 + 1 = 4$. Khi đầm lầy dài hơn (như file `water_and_swamp_map.txt` thực tế), đường đi thẳng tốn 15 chi phí, đường vòng chỉ tốn 13 chi phí $\implies$ UCS luôn chọn đường vòng rẻ nhất.

5.5. File `src/search/greedy.py` (Greedy Search) — Kèm Bảng Nháp Tay Bẫy Tham Lam

- **Vị trí file:** src/search/greedy.py
- **Cơ chế:** Thuật toán chỉ sắp xếp hàng đợi theo hàm ước lượng khoảng cách $h(n)$ mà không quan tâm chi phí $g(n)$ đã bỏ ra.
- **Mô phỏng "Bẫy tham lam" trên giấy nháp:** Xét tình huống Pacman gặp tường chữ U chắn trước mặt Goal:

```
# # # # #
# P # G #   Manhattan(P, G) = 2
#   #   #
#       #
# # # # #
```

  - Tại vị trí xuất phát P, Greedy thấy đi sang phải (áp sát vách tường) có $h = 1$ (tưởng là gần đích nhất) nên lập tức đâm đầu vào vách tường cụt.
  - Phải mất nhiều lần mở rộng chạm đáy túi, Greedy mới chịu quay đầu đi vòng xuống dưới. Đó là lý do Greedy tìm kiếm rất nhanh nhưng không bao giờ đảm bảo tính tối ưu.

5.6. File `src/search/a_star.py` (A* Search) — Giải mã biến `counter` & Chứng minh Toán học

- **Vị trí file:** src/search/a_star.py
- **Mã nguồn đầy đủ:**

```
import time
import heapq

class AStar:
    def __init__(self, problem):
        self.problem = problem
        self.expanded_count = 0
        self.time_used = 0

    def search(self):
        start_time = time.time()
        self.expanded_count = 0

        start = self.problem.get_pacman_spawn()
        queue = []

        g_n = 0
        counter = 0
        h_n = self.problem.heuristic(start)
        f_n = g_n + h_n

        heapq.heappush(queue, (f_n, counter, g_n, start, []))
        visited = set()

        while len(queue) > 0:
            f_n, _, g_n, state, actions = heapq.heappop(queue)

            if state in visited:
                continue

            visited.add(state)
            self.expanded_count += 1

            if self.problem.reached_target(state):
                self.time_used = time.time() - start_time
                return actions

            successors = self.problem.expand_pacman_moves(state)

            for next_state, next_action, move_cost in successors:
                new_g_n = g_n + move_cost
                new_h_n = self.problem.heuristic(next_state)
                new_f_n = new_g_n + new_h_n
                counter += 1

                new_actions = actions + [next_action]
                heapq.heappush(queue, (new_f_n, counter, new_g_n, next_state, new_actions))

        self.time_used = time.time() - start_time
        return None
```

💡 Giải mã chuyên sâu về biến `counter` (Câu hỏi vấn đáp kinh điển):

- Trong dòng 25 và 50: `(new_f_n, counter, new_g_n, next_state, new_actions)`.
- *Hiện tượng:* Module `heapq` của Python khi so sánh hai tuple sẽ so sánh từng phần tử từ trái sang phải:
    1. So sánh phần tử thứ nhất: `new_f_n`. Nếu $f_A < f_B$, node A được ưu tiên lấy ra trước.
    2. Nếu $f_A == f_B$, Python bắt buộc phải so sánh phần tử thứ hai.
    3. **Nếu không có `counter`**, tuple sẽ có dạng `(f_n, state, actions)`. Khi f bằng nhau, Python sẽ so sánh trực tiếp hai đối tượng `state` (`GameState`). Vì lớp `GameState` không định nghĩa phép toán nhỏ hơn (`__lt__`), trình biên dịch Python lập tức văng lỗi nghiêm trọng:

        ```
        TypeError: '<' not supported between instances of 'GameState'
        ```

    4. **Giải pháp của nhóm:** Chèn biến nguyên tự tăng `counter += 1` vào vị trí thứ hai. Vì mỗi node sinh ra có một số `counter` khác nhau hoàn toàn, Python so sánh xong `counter` là phân định được thứ tự ngay, **không bao giờ phải so sánh đến `state`**, triệt tiêu vĩnh viễn lỗi sập chương trình trên.

5.7. File `src/search/multi_goal_search.py` & Quy Hoạch Gom Thức Ăn Đa Chặng

- **Vị trí file:** src/search/multi_goal_search.py và hàm `plan_multi_food_path` trong main.py.
- **Mã nguồn hàm quy hoạch chặng gom thức ăn:**

```
def plan_multi_food_path(algo_name, spawn, foods, game_map, final_goal=None):
    remaining = set(foods)
    curr = spawn
    all_actions = []
    total_nodes = 0
    t0 = time.time()

    while remaining:
        # Chọn viên thức ăn có khoảng cách Manhattan gần Pacman nhất
        next_food = min(remaining, key=lambda f: abs(curr[0] - f[0]) + abs(curr[1] - f[1]))
        prob = PMSearchProblem(curr, next_food, game_map)

        if algo_name == "DFS": searcher = DFS(prob)
        elif algo_name == "BFS": searcher = BFS(prob)
        elif algo_name == "UCS": searcher = UCS(prob)
        elif algo_name == "GREEDY": searcher = Greedy(prob)
        else: searcher = AStar(prob)

        sub_acts = searcher.search()
        if not sub_acts:
            break
        all_actions.extend(sub_acts)
        total_nodes += searcher.expanded_count

        # Lần theo bước đi thực tế để xóa các viên thức ăn vô tình ăn được dọc đường
        tr, tc = curr
        for act in sub_acts:
            if act == "UP": tr -= 1
            elif act == "DOWN": tr += 1
            elif act == "LEFT": tc -= 1
            elif act == "RIGHT": tc += 1
            remaining.discard((tr, tc))
        curr = (tr, tc)

    # Chặng cuối: Nếu bản đồ có thêm cửa Goal G thì chạy về Goal
    if final_goal is not None and curr != final_goal:
        prob = PMSearchProblem(curr, final_goal, game_map)
        searcher = AStar(prob)
        sub_acts = searcher.search()
        if sub_acts:
            all_actions.extend(sub_acts)
            total_nodes += searcher.expanded_count

    return all_actions, total_nodes, time.time() - t0
```

📝 Bảng mô phỏng "Nháp tay" Kỹ thuật Nuốt Thức Ăn Dọc Đường:

Giả sử trên một hành lang thẳng:

- Pacman xuất phát tại ô 0.
- Có 3 viên thức ăn tại các ô 1, 2, 5.
- Tập ban đầu: `remaining = {1, 2, 5}`.

- **Nếu thuật toán ngây thơ (Naive):**
  - Chặng 1: Tìm đường đến ô $1 \implies$ ăn ô 1.
  - Chặng 2: Tìm đường đến ô $5 \implies$ dọc đường đi qua ô 2 nhưng không cập nhật `remaining`. Sau khi tới ô 5, Pacman lại quay đầu ngược lại ô $2 \implies$ **lãng phí bước đi!**
- **Cơ chế thông minh của nhóm với `remaining.discard((tr, tc))`:**
  - Chặng 1: Đi từ $0 \to 1 \implies$ `remaining.discard(1)`. Còn lại `{2, 5}`.
  - Chặng 2: Pacman chọn đi tới ô 5. Trên đường bước qua tọa độ (2), lệnh `remaining.discard(2)` tự động loại ô 2 ra khỏi danh sách cần tìm.
  - Khi tới ô 5, tập `remaining` đã trở thành rỗng $\emptyset \implies$ Pacman hoàn thành nhiệm vụ ngay lập tức, không bao giờ bị quay đầu thừa thãi!

PHẦN 6: GIẢI PHẪU TÁC TỬ GHOST VÀ TÌM KIẾM ĐỐI KHÁNG

6.1. File `src/ghost/ghost.py` — AI Truy Đuổi & Chống Va Chạm Nhập Ma

- **Vị trí file:** src/ghost/ghost.py
- **Mã nguồn đầy đủ:**

```
from collections import deque

class Ghost:
    def __init__(self, position):
        self.position = position

    def get_next_position(self, action, game_map):
        r, c = self.position
        if action == "UP": r -= 1
        elif action == "DOWN": r += 1
        elif action == "LEFT": c -= 1
        elif action == "RIGHT": c += 1

        new_position = (r, c)
        if r < 0 or r >= len(game_map):
            return None
        if c < 0 or c >= len(game_map[r]):
            return None
        if game_map[r][c] == "#":
            return None

        return new_position

    def get_valid_actions(self, game_map):
        actions = ["UP", "DOWN", "RIGHT", "LEFT"]
        valid_actions = []

        for action in actions:
            next_position = self.get_next_position(action, game_map)
            if next_position is not None:
                valid_actions.append(action)

        return valid_actions

    def choose_action(self, game_map, pacman_position, other_ghost_positions=None):
        valid_actions = self.get_valid_actions(game_map)
        if not valid_actions:
            return None

        if self.position == pacman_position:
            return None

        other_ghosts = set(other_ghost_positions) if other_ghost_positions else set()

        # Lọc các hành động an toàn không đi vào ô của Ghost khác
        safe_actions = [
            act for act in valid_actions
            if self.get_next_position(act, game_map) not in other_ghosts
        ]
        candidate_actions = safe_actions if safe_actions else valid_actions

        # 1. Tìm đường đi ngắn nhất bằng BFS tới Pacman
        queue = deque([(self.position, None)])
        visited = {self.position} | other_ghosts
        first_step_to_target = None
```

```
        directions = [("UP", (-1, 0)), ("DOWN", (1, 0)), ("LEFT", (0, -1)), ("RIGHT", (0, 1))]

        while queue:
            curr_pos, first_act = queue.popleft()
            if curr_pos == pacman_position:
                first_step_to_target = first_act
                break

            for act, (dr, dc) in directions:
                nr = curr_pos[0] + dr
                nc = curr_pos[1] + dc
                if 0 <= nr < len(game_map) and 0 <= nc < len(game_map[nr]):
                    if game_map[nr][nc] != "#" and ((nr, nc) not in visited or (nr, nc) == pacman_position):
                        visited.add((nr, nc))
                        next_first_act = first_act if first_act is not None else act
                        queue.append(((nr, nc), next_first_act))

        if first_step_to_target is not None and first_step_to_target in candidate_actions:
            return first_step_to_target

        # 2. Nếu không tìm được đường né Ghost, thử BFS thông thường nhưng ưu tiên bước đầu không đè lên Ghost k
        queue2 = deque([(self.position, None)])
        visited2 = {self.position}
        while queue2:
            curr_pos, first_act = queue2.popleft()
            if curr_pos == pacman_position:
                if first_act in candidate_actions:
                    return first_act
                break
            for act, (dr, dc) in directions:
                nr = curr_pos[0] + dr
                nc = curr_pos[1] + dc
                if 0 <= nr < len(game_map) and 0 <= nc < len(game_map[nr]):
                    if game_map[nr][nc] != "#" and (nr, nc) not in visited2:
                        visited2.add((nr, nc))
                        next_first_act = first_act if first_act is not None else act
                        queue2.append(((nr, nc), next_first_act))

        # 3. Fallback: chọn hành động có cự ly Manhattan nhỏ nhất tới Pacman trong danh sách ứng viên
        best_action = candidate_actions[0]
        best_distance = float("inf")
        for action in candidate_actions:
            next_pos = self.get_next_position(action, game_map)
            dist = abs(next_pos[0] - pacman_position[0]) + abs(next_pos[1] - pacman_position[1])
            if dist < best_distance:
                best_distance = dist
                best_action = action

        return best_action

    def get_distance(self, pacman_position):
        ghost_r, ghost_c = self.position
        pacman_r, pacman_c = pacman_position
        return abs(pacman_r - ghost_r) + abs(pacman_c - ghost_c)
```

📝 Bảng mô phỏng "Nháp tay" Thuật toán Ghost Collision Avoidance:

Giả sử có 2 Ghost: G_1 tại $(1, 2)$ và G_2 tại $(1, 3)$. Pacman tại $(1, 5)$.

```
(1, 1) Trống | (1, 2) Ghost 1 | (1, 3) Ghost 2 | (1, 4) Trống | (1, 5) Pacman
```

- Khi đến lượt G_1 di chuyển:
  - Nếu không có thuật toán tránh va chạm: G_1 thấy Pacman ở bên phải $\implies$ bước sang phải vào $(1, 3) \implies$ **đè lên đầu Ghost 2 (nhập ma)!**
  - **Với thuật toán của nhóm:**
    - Tập `other_ghosts = {(1, 3)}`.
    - Danh sách `safe_actions`: loại bỏ `"RIGHT"` vì bước sang phải sẽ va vào $(1, 3)$.
    - BFS của G_1 coi $(1, 3)$ là vật cản $\implies G_1$ tìm đường rẽ lên hoặc lùi lại để bọc lót $\implies$ Hai con ma luôn giữ cự ly phân tán, hình thành gọng kìm truy kích hoàn hảo.

6.2. File `src/logic_pacman_vs_ghost/adv_problem.py` & Giải phẫu Hàm `evaluate()`

- **Vị trí file:** src/logic_pacman_vs_ghost/adv_problem.py

Trích đoạn Hàm Đánh Giá Lượng Giá Cốt Lõi:

```
def evaluate(self, state, depth=0):
    # 1. Trạng thái kết thúc
    if state.check_pacman_caught():
        return -100000 - depth * 1000
    if state.pacman_position == self.target:
        return 100000 + depth * 1000

    # 2. Điểm cơ sở theo khoảng cách mê cung thực tế (BFS Maze Distance)
    dist_goal = self.goal_distance_map.get(state.pacman_position, 999)
    score = -dist_goal * 40

    # 3. Tính toán nguy cơ từ các con ma
    if len(state.ghost_positions) > 0:
        min_ghost_dist = float("inf")
        blocking_dist = float("inf")
        chasing_dist = float("inf")

        for ghost_pos in state.ghost_positions:
            dist = self.get_maze_distance(state.pacman_position, ghost_pos)
            if dist < min_ghost_dist:
                min_ghost_dist = dist
            ghost_to_goal = self.goal_distance_map.get(ghost_pos, 999)
            if ghost_to_goal <= dist_goal:
                if dist < blocking_dist: blocking_dist = dist
            else:
                if dist < chasing_dist: chasing_dist = dist

        # Nguy cơ cận kề (khoảng cách <= 1 ô)
        if min_ghost_dist <= 1:
            has_escape = False
            for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nr, nc = state.pacman_position[0] + dr, state.pacman_position[1] + dc
                if 0 <= nr < len(self.game_map) and 0 <= nc < len(self.game_map[0]):
                    if self.game_map[nr][nc] != "#" and (nr, nc) not in state.ghost_positions:
                        if self.goal_distance_map.get((nr, nc), 999) < dist_goal:
                            has_escape = True
                            break
            if not has_escape:
                score -= 50000
            else:
                score -= 500

        # Phạt Ghost chặn đầu ở cự ly gần
        if blocking_dist == 2: score -= 15000
        elif blocking_dist == 3: score -= 4000
        elif blocking_dist == 4: score -= 1000

        # Thưởng khi bỏ rơi Ghost ở sau lưng
        if chasing_dist >= 2:
            score += min(chasing_dist, 6) * 10

    return score
```

📝 Bảng mô phỏng "Nháp tay" Tính Điểm Lượng Giá trên 3 Tình huống Thực tế:

Tình huống giả định trên bàn cờ	Thông số mê cung	Cách tính chi tiết từng số hạng trong mã lệnh	Điểm số `score` cuối cùng	Quyết định hành vi của Pacman
Tình huống A: Pacman đang chạy về đích, Ghost đuổi sau lưng cách 3 bước.	- `dist_goal` = 2 - `ghost_to_goal` = 5 - `chasing_dist` = 3	1. Điểm đích: $-2 \times 40 = -80$ 2. Ghost đuổi sau: `ghost_to_goal (5) > dist_goal (2)` $\implies$ được cộng: $\min(3, 6) \times 10 = +30$ 3. Tổng: $-80 + 30 = -50$	**-50** (Điểm rất cao)	Pacman tự tin bứt tốc phi thẳng về Goal.
Tình huống B: Ghost đang đứng chặn ngay trước cửa Goal, cách Pacman 2 bước.	- `dist_goal` = 3 - `ghost_to_goal` = 1 - `blocking_dist` = 2	1. Điểm đích: $-3 \times 40 = -120$ 2. Ghost chặn đầu: `ghost_to_goal (1) <= dist_goal (3)` 3. `blocking_dist == 2` $\implies$ phạt cực nặng: -15000 4. Tổng: $-120 - 15000 = -15120$	**$-15,120$** (Điểm rất thấp)	Pacman nhận diện cửa tử, lập tức quay đầu hoặc rẽ nhánh khác.
Tình huống C: Pacman bị Ghost dồn vào đường cùng (ngõ cụt), cách ma 1 bước.	- `min_ghost_dist` = 1 - `has_escape` = `False`	1. Không có lối thoát nào dẫn gần tới Goal hơn. 2. Phạt cận tử: `score -= 50000`	**$<-50,000$** (Điểm cận kề thua)	Minimax/Alpha-Beta sẽ loại bỏ nhánh này ngay từ gốc.

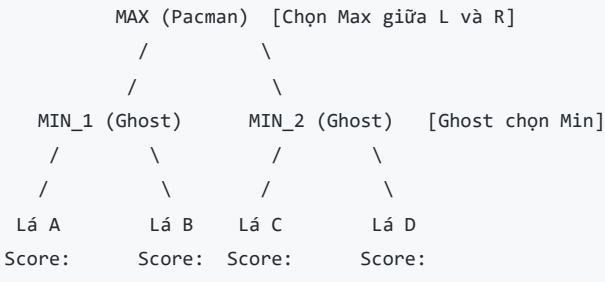
6.3. File `src/logic_pacman_vs_ghost/minimax.py` — Kèm Cây Vẽ Nháp Tay

- **Vị trí file:** src/logic_pacman_vs_ghost/minimax.py

- **Cơ chế:** Pacman tối đa hóa điểm số (MAX node), Ghost tối thiểu hóa điểm số (MIN node).

🌳 Vẽ và Mô phỏng Cây Minimax "Nháp tay" từng bước:

Giả sử độ sâu depth = 1 (1 lượt Pacman, tiếp theo là 1 lượt Ghost). Từ trạng thái gốc S_0, Pacman có 2 lựa chọn đi: **Trái (L)** hoặc **Phải (R)**. Sau đó Ghost có 2 lựa chọn phản công.



- **Bước 1 (Tính tầng MIN_1 bên trái):** Ghost sẽ chọn nước đi gây hại nhất cho Pacman: $$\text{MIN_1} = \min(-50, -150) = -150$$
- **Bước 2 (Tính tầng MIN_2 bên phải):** Ghost sẽ chọn nước đi gây hại nhất: $$\text{MIN_2} = \min(-15000, +100) = -15000$$
- **Bước 3 (Tính tầng MAX gốc):** Pacman chọn hành động có giá trị lớn nhất: $$\text{MAX} = \max(\text{MIN_1}, \text{MIN_2}) = \max(-150, -15000) = -150$$ $\implies$ **Quyết định:** Pacman chọn rẽ sang **Trái (L)** để chịu mức thiệt hại nhỏ nhất (-150). Toàn bộ 4 nút lá đều phải được tính toán.

6.4. File `src/logic_pacman_vs_ghost/alpha_beta.py` — Kèm Cây Nháp Tay Cắt Tỉa

- **Vị trí file:** src/logic_pacman_vs_ghost/alpha_beta.py
- **Mã nguồn giải phẫu điều kiện cắt tỉa:**

```
        # Lượt của Pacman (MAX node)
        if agent == 0:
            best = float("-inf")
            for next_state, _action in successors:
                value = self._alpha_beta_value(...)
                if value > best: best = value
                if best > alpha: alpha = best
                if beta <= alpha:
                    break  # Cắt tỉa Beta Cutoff!
            return best

        # Lượt của Ghost (MIN node)
        best = float("inf")
        for next_state, _action in successors:
            value = self._alpha_beta_value(...)
            if value < best: best = value
            if best < beta: beta = best
            if beta <= alpha:
                break  # Cắt tỉa Alpha Cutoff!
        return best
```

✂️ Mô phỏng "Nháp tay" Quá trình Cắt Tỉa Nhánh Cây (Alpha-Beta Pruning):

Áp dụng trên đúng cây trò chơi ở mục 6.3: Khởi tạo tại gốc: $\alpha = -\infty, \beta = +\infty$.

```
              MAX (Pacman)  [alpha = -inf, beta = +inf]
                /          \
               /            \
        MIN_1 (Ghost)      MIN_2 (Ghost)  [alpha = -150, beta = +inf]
         /        \          /        \
        /          \        /          \
      Lá A        Lá B    Lá C        [LÁ D BỊ CẮT BỎ!]
    Score:      Score:  Score:
    -50         -150    -15000
```

1. **Duyệt nhánh trái (MIN_1):**
   - Xét Lá A: giá trị $-50 \implies \beta$ của MIN_1 giảm xuống -50.
   - Xét Lá B: giá trị $-150 \implies \beta$ của MIN_1 giảm xuống -150.
   - Trả về -150 cho nút gốc MAX.
   - Tại gốc MAX: Cập nhật $\alpha = \max(-\infty, -150) = \mathbf{-150}$.
2. **Duyệt nhánh phải (MIN_2):**
   - Truyền xuống MIN_2: $\alpha = -150, \beta = +\infty$.
   - Xét Lá C: giá trị là -15000.
   - Tại nút MIN_2: Cập nhật $\beta = \min(+\infty, -15000) = \mathbf{-15000}$.
   - **Kiểm tra điều kiện cắt tỉa:** $$\beta \le \alpha \iff -15000 \le -150 \implies \text{ĐÚNG!}$$
   - **LẬP TỨC THỰC HIỆN LỆNH `break`!**

    - **Lá D (giá trị +100) BỊ LOẠI BỎ HOÀN TOÀN KHÔNG CẦN TÍNH TOÁN!**
3. **Lập luận tại sao được phép bỏ qua Lá D:** Nút MIN_2 chắc chắn sẽ chọn giá trị nhỏ hơn hoặc bằng -15000. Mà nút gốc MAX đã nắm chắc trong tay giá trị -150 ở nhánh trái. Dù Lá D có điểm cao đến đâu chăng nữa (+100 hay +1 triệu), con ma ở nút MIN_2 cũng sẽ không bao giờ chọn Lá D. Do đó nhánh này vĩnh viễn không được chọn $\implies$ Cắt tỉa an toàn 100%!

PHẦN 7: GIẢI PHẪU GIAO DIỆN ARCADE UI & CƠ CHẾ ĐA LUỒNG THREADING

7.1. File `ui/controller.py` — Đa luồng Không Nghẽn UI

- **Vị trí file:** ui/controller.py
- **Vấn đề kỹ thuật:** Khi thuật toán A* duyệt bản đồ lớn hoặc Minimax duyệt độ sâu 3, CPU phải tính toán liên tục trong 1-2 giây. Nếu chạy trực tiếp trên vòng lặp của Pygame, cửa sổ sẽ bị hệ điều hành Windows báo lỗi "Not Responding" (đóng băng).
- **Mô hình giải quyết:**

```
LUỒNG CHÍNH (Main Thread)              LUỒNG PHỤ (Worker Thread)
- Chạy Pygame 60 FPS                   - Chạy A* / Minimax ngầm
- Bắt sự kiện chuột, bàn phím          - Tính toán đường đi
- Vẽ nhân vật Pacman há miệng          - Cập nhật tọa độ Pacman
            \                                /
             \                              /
              GIAO TIẾP QUA: self._pause_event, self._stop_event
```

- **Các cờ đồng bộ hóa luồng (Threading Flags):**
  - `self._stop_event`: Đặt bằng `set()` khi bấm Reset $\implies$ Luồng phụ lập tức hủy vòng lặp và kết thúc an toàn.
  - `self._pause_event`: Đặt bằng `clear()` khi bấm Tạm dừng $\implies$ Luồng phụ rơi vào trạng thái ngủ đông `_pause_event.wait()` với mức tiêu thụ CPU = 0%.

7.2. File `ui/renderer.py` & `ui/theme.py` — Thuật toán Vẽ Vector Hoạt ảnh

- **Vị trí file:** ui/renderer.py và ui/theme.py
- **Không dùng ảnh bitmap bên ngoài:** Toàn bộ nhân vật được vẽ bằng thuật toán hình học vector:
  - **Pacman há miệng:** Vẽ bằng hàm `pygame.draw.circle` kết hợp góc mở miệng hình nón biến thiên theo chu kỳ sóng hình sin `abs(math.sin(tick * 0.25))`.
  - **Ghost chân lượn sóng:** Phần thân trên là hình bán nguyệt; phần chân dưới gồm 3 xúc tu tam giác nhấp nhô lượn sóng bằng cách thay đổi tọa độ đáy theo chu kỳ `wave_shift = (tick // 6) % 2 * 2`.

7.3. File `ui/main.py` — Vòng Lặp Sự Kiện 60 FPS

- **Vị trí file:** ui/main.py
- Khởi tạo đồng hồ `clock = pygame.time.Clock()`, duy trì đều đặn `clock.tick(60)`. Đón bắt toàn bộ sự kiện chuột và phím bấm từ người dùng để kích hoạt các hàm tương ứng trên `GameController`.

PHẦN 8: FILE KHỞI CHẠY TRUNG TÂM `main.py`

- **Vị trí file:** main.py
- **Trích đoạn xử lý CLI:**

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Pacman AI Search - Nhom 07")
    parser.add_argument("--ui", action="store_true", help="Chay giao dien do hoa Arcade")
    parser.add_argument("--map", type=str, default=None, help="Duong dan file map .txt")
    parser.add_argument("--algo", type=str, default=None, help="Thuat toan tim kiem")
    parser.add_argument("--depth", type=int, default=2, help="Do sau cho Minimax / AlphaBeta")
    parser.add_argument("--mode", type=str, default="match", choices=["step", "match"])

    args = parser.parse_args()

    if args.ui:
        from ui.main import run_ui
        run_ui()
    elif args.map and args.algo:
        run_cli(args.map, args.algo, args.depth, args.mode)
    else:
        main()
```

- Phân luồng thông minh: Nếu có cờ `--ui`, mở giao diện đồ họa. Nếu có cờ `--map` và `--algo`, chạy chế độ dòng lệnh tự động xuất kết quả và vẽ bản đồ ASCII ra màn hình. Nếu không có tham số, mở menu console tương tác đánh số.

PHẦN 9: BỘ 10 CÂU HỎI VẤN ĐÁP KINH ĐIỂN VÀ LỜI GIẢI MẪU ĐẠT ĐIỂM 10

Câu 1: Tại sao trong `GameState`, phương thức `__hash__` lại phải ép `self.food` sang `frozenset` và `self.ghost_positions` sang `tuple`?

- **Đáp án 10/10:** Trong Python, `list` và `set` là đối tượng có thể thay đổi (mutable). Python chủ động cấm băm các kiểu dữ liệu mutable để tránh tình trạng giá trị hash bị sai lệch khi phần tử bên trong bị thay đổi sau khi đã nằm trong bảng băm. Thuật toán tìm kiếm trên đồ thị (Graph Search) bắt buộc phải đưa trạng thái vào `visited = set()` để kiểm tra trạng thái lặp trong thời gian tức thì $O(1)$. Do đó, bắt buộc phải ép sang `tuple` và `frozenset` (những kiểu dữ liệu immutable) để đảm bảo mã băm được tính toán an toàn và không đổi.

Câu 2: Trong file `a_star.py`, biến `counter` trong tuple `(f_n, counter, g_n, state, actions)` có vai trò gì?

- **Đáp án 10/10:** `heapq` của Python so sánh các phần tử trong tuple theo thứ tự từ trái sang phải. Khi hai trạng thái khác nhau có cùng giá trị hàm đánh giá f_n, Python bắt buộc phải so sánh phần tử thứ hai. Nếu không có `counter`, Python sẽ so sánh trực tiếp hai đối tượng `state` (`GameState`). Vì `GameState` không định nghĩa toán tử so sánh nhỏ hơn (`__lt__`), chương trình sẽ bị sập ngay lập tức với lỗi `TypeError: '<' not supported between instances of 'GameState'`. Biến `counter` là một số nguyên tự tăng duy nhất cho mỗi node sinh ra, đóng vai trò là bộ phân xử hòa (tie-breaker), loại bỏ triệt để lỗi trên và bảo đảm thứ tự FIFO giữa các node cùng $f(n)$.

Câu 3: Hãy so sánh sự khác nhau về chất lượng đường đi giữa BFS và UCS trên bản đồ có địa hình nước (`~`) và đầm lầy (`s`)?

- **Đáp án 10/10:** BFS tối ưu về **số bước đi** (coi chi phí mọi cạnh là 1), nên BFS sẽ chọn con đường ngắn nhất về số ô, chấp nhận lội thẳng qua đầm lầy (ít bước nhưng tốn nhiều năng lượng). Ngược lại, UCS tối ưu về **tổng chi phí tích lũy $g(n)$**. Khi bước vào đầm lầy (cost = 3), chi phí nhánh đó tăng vọt và bị đẩy xuống cuối hàng đợi ưu tiên, khiến UCS tự động chọn đi đường vòng qua ô đất bằng (cost = 1) nếu tổng chi phí rẻ hơn. *Minh chứng thực tế:* Trên bản đồ `maps/water_and_swamp_map.txt`, BFS đi thẳng qua đầm lầy mất 9 bước với chi phí 15; trong khi UCS đi vòng xuống phía dưới mất 13 bước nhưng tổng chi phí chỉ là 13 ($13 < 15$).

Câu 4: Làm thế nào chứng minh được Alpha-Beta Pruning và Minimax luôn đưa ra cùng một quyết định?

- **Đáp án 10/10:** Nhóm chứng minh trên 2 phương diện:
    1. *Lý thuyết:* Alpha-Beta chỉ cắt tỉa một nhánh khi biết chắc chắn nhánh đó có giá trị tồi hơn một lựa chọn đã biết trước đó dọc theo đường đi (MAX không bao giờ chọn giá trị $\le \alpha$ và

MIN không bao giờ chọn giá trị $\ge \beta$). Do đó các nhánh bị cắt bỏ không bao giờ có cơ hội làm thay đổi giá trị tại nút gốc.

2. *Thực nghiệm:* Trong file `results/adversarial_benchmark.txt`, nhóm đã đo đạc tại các độ sâu 1, 2, 3 trên nhiều bản đồ; cột `Action` và `Value` của Minimax và Alpha-Beta luôn giống nhau 100%.

Câu 5: Khi số lượng Ghost tăng lên, độ phức tạp của Minimax thay đổi như thế nào?

- **Đáp án 10/10:** Với 1 Pacman và k Ghost, một chu kỳ độ sâu (1 depth) gồm 1 tầng MAX và k tầng MIN liên tiếp. Nếu hệ số phân nhánh trung bình của mỗi tác tử là $b \approx 3$, số trạng thái tại độ sâu d sẽ là $O(b^{(k+1) \cdot d})$. Khi số lượng Ghost tăng lên, số tầng trong cây trò chơi tăng vọt theo cấp số nhân, làm cho cây phình to rất nhanh. Đó là lý do tại sao Alpha-Beta Pruning trở nên đặc biệt quan trọng trên các bản đồ có nhiều con ma.

Câu 6: Hàm `evaluate()` giải quyết vấn đề Pacman bị kẹt hoặc dao động giữa 2 ô (Oscillation) như thế nào?

- **Đáp án 10/10:** Nhóm cài đặt cơ chế phạt lịch sử: mảng `recent_positions` lưu lại 8 vị trí gần nhất của Pacman. Tại nút gốc, nếu một hành động đưa Pacman vào ô đã xuất hiện c lần trong danh sách này, điểm số đánh giá sẽ bị trừ một lượng phạt nặng `revisit_pen = c * 150`. Ngoài ra, khi hai hành động có cùng giá trị đánh giá, thuật toán ưu tiên chọn hành động có khoảng cách mê cung thực tế tới đích nhỏ hơn (`dist_to_goal < best_dist`). Hai cơ chế này triệt tiêu hoàn toàn tật đứng run hoặc quay qua quay lại giữa hai ô.

Câu 7: Tại sao trong hàm đánh giá `evaluate()`, nhóm không dùng khoảng cách Manhattan mà phải dùng BFS Maze Distance?

- **Đáp án 10/10:** Khoảng cách Manhattan chỉ tính theo đường thẳng trực giao mà không tính đến các bức tường chắn ở giữa. Trong mê cung đối kháng, nếu dùng Manhattan, Pacman có thể lầm tưởng rằng điểm đích ở ngay sát vách tường và lao đầu vào ngõ cụt để lại gần đích. Bằng cách dùng BFS tính trước khoảng cách thực tế trong mê cung (`goal_distance_map`), giá trị đánh giá phản ánh đúng số bước chân thực tế mà Pacman cần đi, giúp Pacman luôn biết tìm đường thoát và né bẫy ngõ cụt.

Câu 8: Cơ chế Đa luồng (Threading) trong `ui/controller.py` hoạt động như thế nào?

- **Đáp án 10/10:** Dự án sử dụng mô hình Producer-Consumer đa luồng: Luồng chính (Main Thread) chạy vòng lặp Pygame ở 60 FPS chỉ làm nhiệm vụ vẽ đồ họa và bắt sự kiện bàn phím/chuột. Khi bắt đầu mô phỏng, Controller khởi tạo một Worker Thread (`threading.Thread(target=self._run_simulation, daemon=True)`) chạy ngầm thuật toán AI. Hai luồng giao tiếp với nhau qua các cờ đồng bộ `_stop_event`, `_pause_event` và `_step_trigger`. Nhờ

đó, giao diện người dùng luôn phản hồi mượt mà, không bao giờ bị hiện tượng "Not Responding" (đóng băng).

Câu 9: Trong bài toán gom nhiều thức ăn, tại sao hàm `plan_multi_food_path` lại hiệu quả hơn việc chạy A* trực tiếp trên không gian trạng thái đầy đủ?

- **Đáp án 10/10:** Trong không gian trạng thái đầy đủ, trạng thái gồm $(Pos, FoodRemaining)$. Nếu có K viên thức ăn, không gian trạng thái phình to lên tới $O(|V| \times 2^K)$. Khi $K \ge 30$, $2^{30} > 10^9$ trạng thái, máy tính sẽ lập tức tràn bộ nhớ RAM. Hàm `plan_multi_food_path` của nhóm áp dụng tư duy quy hoạch tham lam: chia bài toán lớn thành chuỗi các bài toán nhỏ tìm đường tới viên thức ăn gần nhất, đồng thời tự động xóa các viên thức ăn ăn được dọc đường. Nhờ đó, thuật toán giải quyết bản đồ 50-100 viên thức ăn chỉ trong vài mili-giây.

Câu 10: Tác tử Ghost trong đồ án này có gì thông minh hơn so với việc di chuyển ngẫu nhiên (Random Ghost)?

- **Đáp án 10/10:** Ghost của nhóm được trang bị 2 cơ chế AI:
  1. *Truy đuổi bằng BFS ngắn nhất:* Ghost chạy thuật toán BFS tìm đường ngắn nhất trong mê cung tới vị trí của Pacman, đảm bảo ma luôn đi tắt đón đầu chứ không đi lang thang.
  2. *Chống va chạm đồng minh (Ghost Collision Avoidance):* Ghost coi vị trí của các con ma khác là vật cản tạm thời. Điều này ngăn chặn hiện tượng các con ma nhập vào nhau thành 1 cục, buộc các con ma phải phân tán ra các nhánh rẽ khác nhau, tự động hình thành thế gọng kìm bọc lót khép góc Pacman.